

Graph Neural Networks in Network Neuroscience

Part 1 (~Section 2.4)

Siwon Yun

BCSC 2025

August 17, 2025

1 introduction

- network neuroscience field
- 3가지 뇌 그래프 유형
- 뇌 연결과 관련된 3가지 축
- geometric and hyperbolic spaces에 뇌 그래프 임베딩하기
- graph neural networks와 relational graph

2 what do graph neural networks offer to network neuroscience?

- GNN overview
- brain graph prediction
 - Cross-domain graph prediction
 - Cross-resolution graph prediction
 - Cross-time graph prediction

introduction

network neuroscience field의 목표(배경)과 방법론

- **목표:** 생물학적 신경 시스템의 기초적인 연결성이 뇌 기능에 어떻게 기여하는지 알아보기
- **방법론:** neuron의 활동을 지도화하기. 즉, 뇌를 network로 modeling하기
 - node → anatomical brain regions
 - edges → morphological, functional, or structural connectivities

뇌를 그래프로 모델링하는 전통적인 3가지 그래프
(뇌를 관찰하는 방법에 의존하여 분류됨)

- **morphological brain graph:** T1-weighted MRI
- **structural brain graph:** diffusion- weighted MRI
- **functional brain graph:** resting-state functional MRI

뇌 연결의 일생(life span)을 이해하는 세가지 축¹ : time, resolution, domain

- **time:**

출생 → 죽음까지 시간에 따른 뇌 연결성 변화.

활용 예:

- 어린 시절 뇌 연결 → 정상적인 뇌 연결: 정상
- 어린 시절 뇌 연결 치매 이전 뇌 연결: 진단 ✗ 치매의 뇌 연결을 희망

- **resolution:**

더 높은 해상도는 새로운 뇌 영역 간의 연결을 발견 가능.

low-resolution vs super-resolution: ROIs²와 edge의 개수 차이. 전자는 node와 edge가 적고 후자는 많음

- **domain:**

데이터를 어떤 분석 기법(즉, 'neuroimaging modality' (e.g., functional MRI or diffusion MRI))으로 얻었는가?

cross-domain multimodal → 상호 보완하여 더 좋은 정보 획득 가능

⇒ 각 축에 관점서 획득한 데이터를 조합하여 multimodal dataset을 만들면 더 풍부한 정보가 될 수 있음: e.g., Human Connectome Project (HCP), Baby Connectome Project (BCP), Connectome Related to Human Disease

⇒ multimodal이 더 풍부한 정보를 줌 → 때문에 2가지 관점(morphological, structural, functional 또는 time, resolution, domain)에서 어떤 요소가 없더라도 추측하는 ML기반 생성모델이 연구되고 있음

¹axis:축, 관점. 예컨대, ' $y = x^2$ 은 x축 관점에서 기하급수적으로 증가하고 있어'

²여기서 ROIs는 관심 영역보다는 뇌를 몇개로 나눌까, 즉 ROIs는 node를 나타냄

geometric and hyperbolic spaces에 뇌 그래프 임베딩하기
임베딩을 위한 다양한 ML model이 개발 되었음

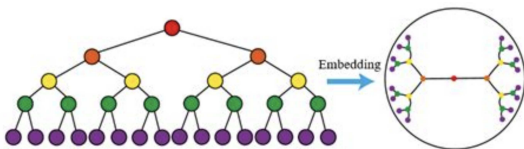
- **BRAINtrinsic:**

초기, 뇌 연결의 topology를 가지고 기하학적 요소 없는 PCV 등 ML의 차원축소 기법을 적용함.

사용한 데이터는 위치적 정보가 포함되어 있으나, 결과적으로 차원축소를 통해 유용한 정보를 얻음. 즉, 결과는 위치적 정보와 관련성이 낮아질 수 있음

- **hyperbolic space에 임베딩:**

뇌 연결성을 hyperbolic space에 매핑함. 거리가 가까울수록 연결될 확률이 높음이 알려짐. 2-D hyperbolic discs에 표현하는 방법이 제안 되었으며, 환자의 경우 비정상적인 경로가 존재하였음



(a) hyperbolic space로 임베딩



(b) hyperbolic disc

Figure 1: hyperbolic space

graph neural networks와 relational graph

- 기존: artificial neural network (ANN)는 순방향 그래프
- → 보완: relational graph, 각 노드간 서로 메시지를 주고 받음
- 문제: 신경 연결과 유사함, 비단 non-Euclidean data types³가 input인 경우 network서 non-Euclidean적 특성을 유지 못함
- → 보완: graph neural networks (GNNs), 학습 중 graph topological 특징을 유지함

⇒ 본 review에서 GNN을 알아보아요!

³예컨대 시계열과 언어는 Euclidean, graph는 non-Euclidean

what do graph neural networks offer to network neuroscience?

non-Euclidean data에 대한 geometric deep learning 분야의 하위 분야
GNN은 그래프 구조를 사용하는 신경망이다. 대개, 인접행렬로 그래프를 표현한다.

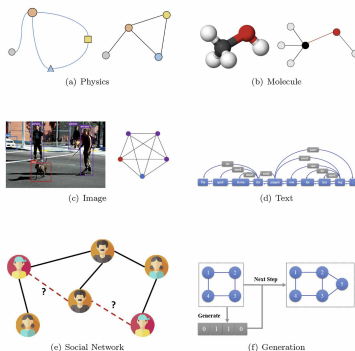


Figure 2: GNN example

GNN의 3가지 카테고리

- !!추후 추가!!

brain connectome $G = (N, E, \mathbf{A}, \mathbf{F})$, $\mathbf{A} \in \mathfrak{M}_{n,n}(\mathbb{R})$, $\mathbf{F} \in \mathfrak{M}_{n,f}(\mathbb{R})$

propagation rule of the GCN model:

$$\mathbf{Z} = f_{\phi}(\mathbf{F}, \mathbf{A} \mid \Theta) = \phi(\overbrace{\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}}^{\text{Laplacian matrix}} \mathbf{F} \Theta) \quad (1)$$

수식의 요소

- N : set of nodes (i.e, ROIs or subjects)
- E : set of edges
- $\mathbf{A}$: 각 노드간 관계를 나타내는 connectivity matrix, 즉 인접 행렬(adjacency matrix)임⁵
- $\mathbf{F}$: 노드의 특성을 나타내는 행렬. 뇌 영역에 대한 자연스러운 특징은 없어서 identity matrix로 초기화함
- ϕ : activation function. e.g., ReLU
- $\mathbf{Z}$: propagation된 layer의 결과
- $f_{(\cdot)}$: graph convolution function, Equation 1의 최우변이 함수의 정의
- Θ : graph convolutional weights를 나타내는 filter⁶
- $\tilde{\mathbf{A}}: \mathbf{A} + \mathbf{I}$
- $\tilde{\mathbf{D}}$: diagonal matrix, $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$

⁴Laplacian matrix $D - A$ 의 normalization 버전. 각각은 Degree와 adjacent matrix

⁵노드의 특징? 예컨대 volume 등

⁶선형모델 $y = x^T \theta$ 와 같은 맥락

At the population level

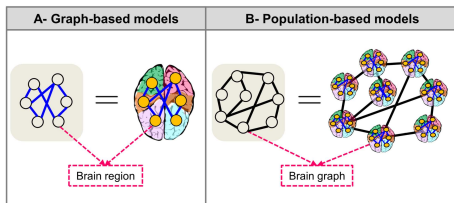


Figure 3: GNN-based methods

- **A:** 뇌간의 서로 비슷한 정도
- **F:** 각 뇌 그래프의 특징

brain graph prediction

뇌 그래프를 예측하는 다양한 GNN 모델을 본 리뷰에서 조사하였다고 함

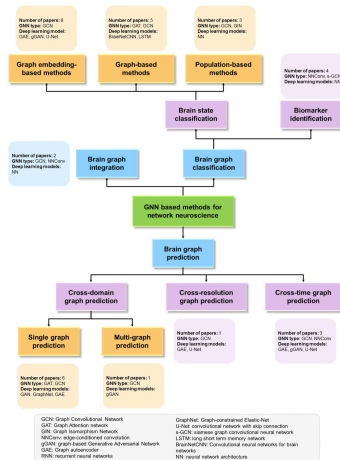


Figure 4: Categorization landscape of GNN-based methods for network neuroscience.

2.4절에서는 brain graph prediction을 다룸

Cross-domain graph prediction

최적의 매핑 함수 $f : (\mathbf{A}_s, \mathbf{F}_s) \mapsto (\mathbf{A}_t, \mathbf{F}_t)$ 를 찾는 것을 목표로 함⁷

e.g., Functional $\mapsto$ Structural

architecture:

- Discriminator: 실제 vs 생성된 그래프 구분 $\rightarrow$ 도메인 정렬 문제

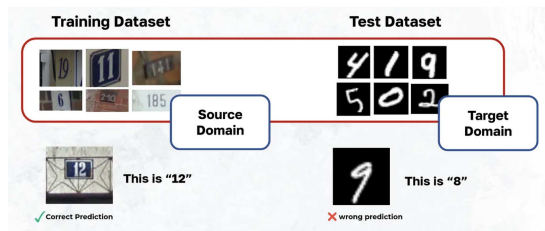


Figure 5: domain shift

- Generator: source graph $\rightarrow$ target graph

$\Rightarrow$ E2E^{edge to edge}, 즉 통합된 하나의 모델로 만들기 어려움: 성능 저하, 복잡성 증가

⁷ML의 궁극적 목표는 현상을 가장 잘 표현하는 매핑을 찾는 것

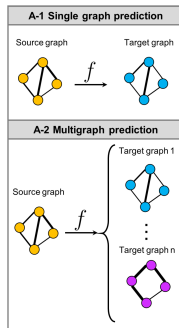


Figure 6: single-graph prediction vs multi-graph prediction

single-graph prediction model과 multi-graph prediction model로 구분 가능
multi-graph prediction은 확장성(보편성)과 학습 효율성이 높은 반면, 제작이 어려움

Cross-resolution graph prediction

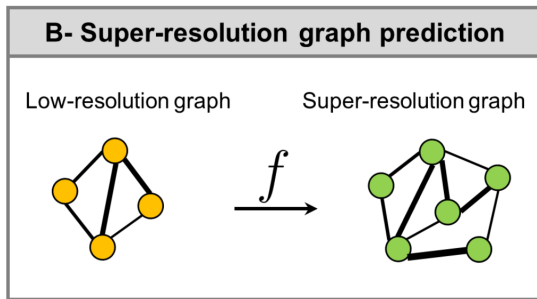


Figure 7: super-resolution graph prediction

$G_l = (N_l, E_l, \mathbf{A}_l, \mathbf{F}_l)$ 과 $G_s = (N_s, E_s, \mathbf{A}_s, \mathbf{F}_s)$, ($M_l < N_s$)에 대하여 매핑 함수 $f : (\mathbf{A}_l, \mathbf{F}_l) \mapsto (\mathbf{A}_s, \mathbf{F}_s)$ 을 찾는 것을 목표로 함
즉, low-resolution graph를 high-resolution graph로 변환하는 것

Cross-time graph prediction

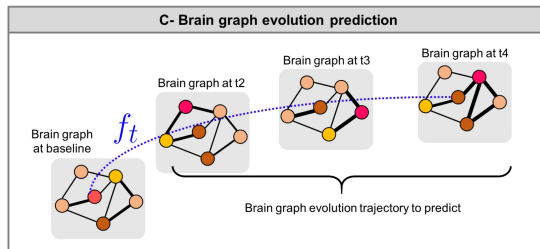


Figure 8: brain graph evolution prediction

시계열 $G_{ti} = (N, E, \mathbf{A}_{ti}, \mathbf{F}_{ti}), i \in \{1, \dots, n_t\}$, evolution trajectory $\tau = \{G_{ti+1}\}_{i=2}^{n_t}$ 에 대하여 매핑함수 $f_t : (\mathbf{A}_{ti}, \mathbf{F}_{ti}) \mapsto (\tau)$ 을 찾는 것을 목표로 함
즉, 시간에 따른 뇌 그래프 변화를 예측함

TABLE 1

Brain graph prediction publications. In the first column, we list the name of existing works along with their citations for easy reference. In the second and third columns, we list the GNN model used and we briefly summarize all losses used to regularize the existing models. In the three last columns, symbols ✓ and ✗ denote whether the corresponding model is learned in an end-to-end fashion (E2E), learned from a brain graph (GM), or a population graph (PM), respectively.

Method	GNN	Loss	E2E	GM	PM
CGTS [61]	GCN ^a	$\mathcal{L} = \mathcal{L}_{gen1}(G_1, D_1) + \mathcal{L}_{gen2}(G_2, D_2) + \alpha_1[\mathcal{L}_{topo}(G_1, D_1) + \mathcal{L}_{topo}(G_2, D_2)] + \alpha_2\mathcal{L}_{cyc}(G_1, G_2)$	✓	✓	✗
MGCN-GAN [44]	GCN	$\mathcal{L} = \mathcal{L}_{GAN} + \alpha\mathcal{L}_{MSE} + \beta\mathcal{L}_{PCC}$	✓	✓	✗
DMBN [62]	MGCK ^b	$\mathcal{L} = \alpha\mathcal{L}_{global} + \beta\mathcal{L}_{local} + \mathcal{L}_{sup}$		✓	✗
HADA [64]	GCN	$\mathcal{L} = \begin{cases} \mathcal{L}_{hier} = \mathbb{E}[\log(D(\mathbf{Z}^{h-1}))] + \mathbb{E}[\log(1 - D(G^{h-1}))] \\ \mathcal{L}_{source} = \mathbb{E}[\log(D(\mathbf{Z}_{src}))] + \mathbb{E}[\log(1 - D(G))] \end{cases}$	✗	✗	✓
SymDADA [65]	GCN	$\mathcal{L} = \begin{cases} \mathcal{L}_{sym} = \mathbb{E}[\log(D_1(\mathbf{Z}_{DA}))] + \mathbb{E}[\log(1 - D_1(G))] \\ \mathcal{L}_{dual1} = \mathbb{E}[\log(D_1(\mathbf{Z}_{src}))] + \mathbb{E}[\log(1 - D_1(G))] \\ \mathcal{L}_{dual2} = \mathbb{E}[\log(D_2(\mathbf{F}))] + \mathbb{E}[\log(1 - D_2(\hat{\mathbf{F}}))] \end{cases}$	✗	✗	✓
LG-DADA [63]	GCN	$\mathcal{L} = \begin{cases} \mathcal{L}_{align} = \mathbb{E}[\log(D_1(\mathbf{F}_s))] + \mathbb{E}[\log(1 - D_1(\mathbf{Z}_t))] \\ \mathcal{L}_{dual_a} = \mathbb{E}[\log(D_1(\mathbf{Z}_s))] + \mathbb{E}[\log(1 - D_1(\mathbf{F}_s))] \\ \mathcal{L}_{pred_b} = \mathbb{E}[\log(D_2(\mathbf{F}_t))] + \mathbb{E}[\log(1 - D_2(\hat{\mathbf{F}}_t))] \end{cases}$	✗	✗	✓
MultiGraphGAN [66]	GCN	$\mathcal{L} = \begin{cases} \mathcal{L}_D = \sum_{j=1}^c (\mathcal{L}_{WD}^j + \alpha_1\mathcal{L}_{gcd}^j + \alpha_2\mathcal{L}_{gp}^j) \\ \mathcal{L}_G = \sum_{j=1}^c (\mathcal{L}_{gen3}^j + \alpha_1\mathcal{L}_{topo2}^j + \alpha_2\mathcal{L}_{rec}^j + \alpha_3\mathcal{L}_{info}^j) \end{cases}$	✗	✗	✓
GSR-Net [70]	GCN	$\mathcal{L} = \mathcal{L}_{hr} + \mathcal{L}_{eig} + \alpha\mathcal{L}_{rec}$	✓	✓	✗
RESNet [71]	GCN	$\mathcal{L}_{emb} = \mathbb{E}[\log(D_1(\mathbf{A}_{t_0}))] + \mathbb{E}[\log(1 - D_1(\mathbf{z}_{t_0}))]$	✗	✓	✗
gGAN [72]	ECC ^c	$\mathcal{L} = \mathcal{L}_{adv} + \alpha\mathcal{L}_1(N)$	✗	✓	✗
EvoGraphNet [73]	ECC	$\mathcal{L} = \sum_{i=1}^m (\alpha_1\mathcal{L}_{adv}(G_i, D_i)) + \frac{\alpha_2}{n_s} \sum_{s=1}^{n_s} \mathcal{L}_1(G_i, s) + \frac{\alpha_3}{n_s} \sum_{s=1}^{n_s} \mathcal{L}_{KL}(t_i, s)$	✓	✓	✗

^aGraph convolutional networks; ^bMulti-stage graph convolution kernel; ^cEdge-conditioned convolution

Figure 9: Table 1